Introduction to ReactJS

React vs. plain JavaScript · Vite setup · Project structure · Components · useState · useEffect · Fetching data from an API

---

1. Learning Objectives

In this session you learn **ReactJS**, one of the most widely used JavaScript libraries for building user interfaces, and use it to display and manipulate data that comes from a web API.

After this session, you will be able to:

- Explain how React differs from plain JavaScript (imperative vs. declarative UI) and why teams adopt it.
- Create a React project with **Vite**, run it, and explain the purpose of every generated file.
- Organize a project into components, pages, hooks and services folders.
- Write components with **JSX** and pass data using **props**.
- Manage data that changes over time with **useState**.
- Run side effects (fetching data, timers) with **useEffect** and clean them up.
- Know when to use useRef and useContext, and write a small **custom hook** (useFetch, useDebounce).
- Fetch data from a public REST API with fetch(), show loading and error states, and keep API calls in a services/ file.

2. Why React? React vs. Plain JavaScript

2.1 The Problem with Manual DOM Updates

With plain JavaScript you describe **how** to change the page, step by step: find an element, change its text, add a node, remove another one. As the page grows, you must remember to update every place that shows the same data. A counter in plain JavaScript:

```
<h2 id="count">0</h2>
<button id="btn">+1</button>
```

```
let count = 0;
const label = document.getElementById('count');
const btn = document.getElementById('btn');

btn.addEventListener('click', () => {
  count++;
  label.textContent = count;   // YOU must update the DOM by hand
});
```

React flips the approach. You describe **what the page should look like for the current data** (the *state*), and React figures out which parts of the real DOM to change. This style is called **declarative** programming.

```
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </>
  );
}
```

Key idea: **UI = f(state).** You never touch the DOM. You change the state, and React re-runs your component and updates only what changed.

2.2 Comparison

Aspect	Plain JavaScript	React
Style	Imperative: “find this element, then change it”	Declarative: “given this data, the UI looks like this”
Updating the page	Manual document.getElementById(...), innerHTML	Change state; React updates the DOM
Reuse	Copy–paste HTML and scripts	Components: write once, use many times with different props

Aspect	Plain JavaScript	React
Data ↔ UI sync	Easy to get out of sync in large pages	Always in sync because UI is derived from state
Tooling	Just a <script> tag	Build tool (Vite), npm packages, JSX compilation
Learning curve	Low at first, painful at scale	Higher at first, scales well with team and project size
Good fit	Small static pages, quick demos	Interactive apps, dashboards, admin panels

2.3 Core Ideas of React

- **Component:** a JavaScript function that returns UI. Components can contain other components, forming a tree.
- **JSX:** HTML-like syntax inside JavaScript. A build tool turns it into plain JavaScript calls.
- **Props:** inputs passed *down* from a parent to a child component (read-only).
- **State:** data owned by a component that can change over time; changing it triggers a re-render.
- **Virtual DOM:** React compares the new UI with the previous one in memory and applies only the minimal changes to the real DOM.
- **SPA (Single Page Application):** the browser loads one HTML page once; React updates the screen and calls your API for data, instead of loading a new HTML page for every click.

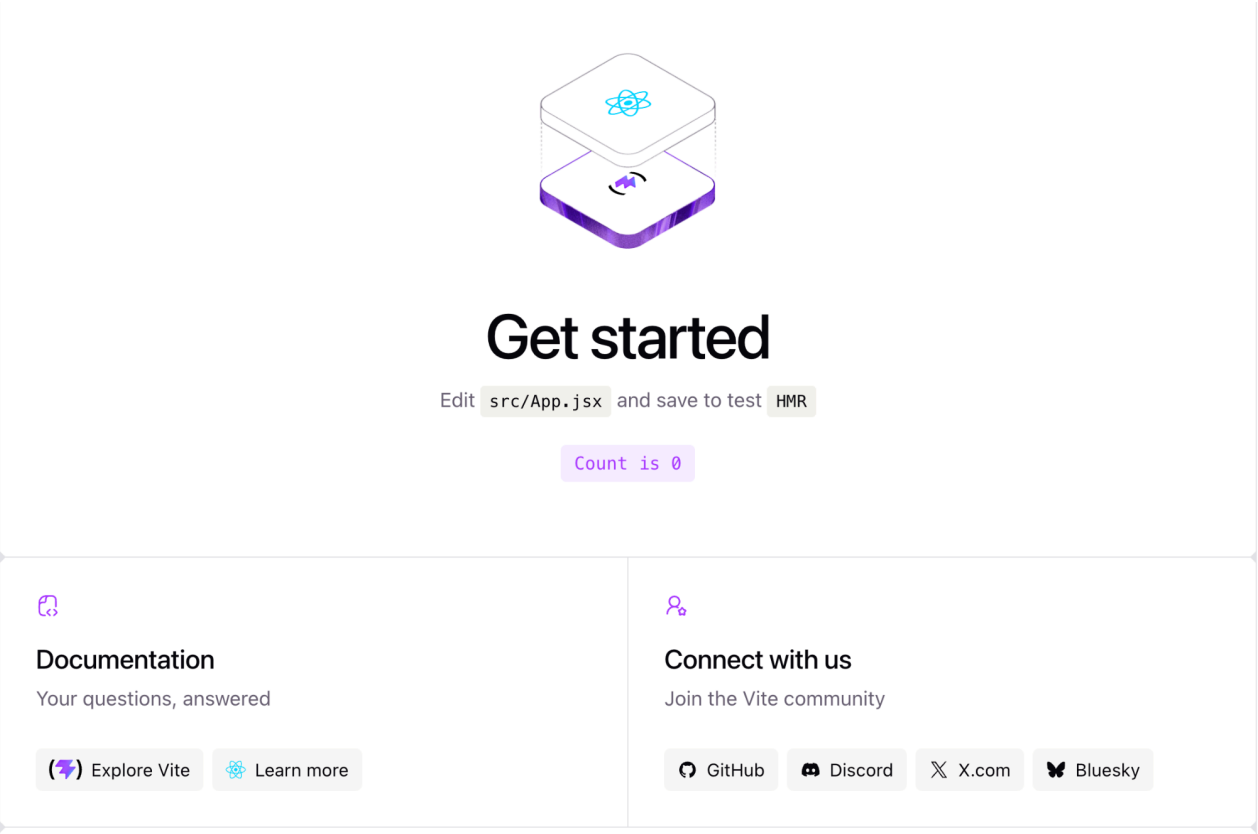
3. Setting Up a React Project

3.1 Create the Project with Vite

Vite is a fast development server and build tool. It replaced the older Create React App, which is no longer recommended.

```
npm create vite@latest product-ui -- --template react
cd product-ui
npm install
npm run dev
```

If the wizard asks additional questions (such as experimental options), accept the defaults. Open the printed address – normally ***http://localhost:5173***. Vite 8 requires **Node.js 20.19+ or 22.12+** – check with node -v before you start.



3.2 What Vite Generated

```
product-ui/
├── node_modules/       (installed packages – never edit, never commit)
├── public/             (static files served as-is, e.g. favicon)
├── src/
│   ├── assets/         (images imported by components)
│   ├── App.jsx         (root component)
│   ├── App.css
│   ├── index.css       (global styles)
│   └── main.jsx        (entry point: mounts <App /> into #root)
├── index.html          (the single HTML page)
├── package.json        (dependencies + npm scripts)
├── vite.config.js      (Vite settings)
├── eslint.config.js    (ESLint code-quality rules)
├── README.md
└── .gitignore
```

File	Purpose
index.html	The only HTML page. Contains and a script tag loading src/main.jsx.
src/main.jsx	Entry point. Finds #root and renders <App /> into it.
src/App.jsx	The root component. Everything on screen descends from it.
package.json	Project metadata, dependencies and scripts: dev, build, preview.
vite.config.js	Vite configuration (plugins, dev-server options).
.env	Environment variables you create. Names exposed to the browser must start with VITE_.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

3.3 Recommended Project Structure

The default template is only a starting point. Create the folders below as soon as your project has more than one screen. The rule of thumb: **components know how to display, pages know what to show, services know how to talk to the server.**

```
src/
├── components/        (small, reusable UI pieces)
│   ├── Navbar.jsx
│   └── ProductCard.jsx
├── pages/             (one component per screen)
│   ├── LoginPage.jsx
│   └── ProductsPage.jsx
├── hooks/             (custom hooks)
│   └── useFetch.js
├── services/          (all backend calls live here)
│   └── api.js
├── App.jsx
├── main.jsx
└── index.css
```

Folder	What goes inside	Rule
components/	Reusable, mostly visual pieces: Navbar, ProductCard, Button	Receive data through props; avoid calling the API directly
pages/	One component per screen: LoginPage, ProductsPage	Combine components, hold page-level state, call hooks/services
hooks/	Custom hooks such as useFetch, useDebounce	File name starts with use
services/	Functions that call the backend (api.js)	The only place that knows URLs, headers and tokens

> **Warning**
> **Naming conventions:** component files and functions use PascalCase (ProductCard.jsx). A component named in lower case (<productCard />) is treated as an HTML tag and will not work.

4. JSX, Components and Props

4.1 JSX Rules

- A component returns **one root element**. Use a fragment <>...</> if you do not want an extra .
- Use { } to embed any JavaScript expression: <h1>Hello, {name}</h1>.
- Attributes use camelCase: className (not class), onClick, htmlFor.

- Every tag must be closed: <input />, <img />.
- Inline styles take an object: style={{ color: 'red', fontSize: 18 }}.

```
export default function App() {
  const name = 'Nguyen Van A';
  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>2 + 3 = {2 + 3}</p>
    </div>
  );
}
```

4.2 Components and Props

Props are the arguments of a component. Destructure them in the parameter list and give optional ones a default value.

```
export default function ProductCard({ name, price, inStock = true }) {
  return (
    <div className="card">
      <h3>{name}</h3>
      <p>{price.toLocaleString()} USD</p>
      {inStock ? <span>In stock</span> : <span>Sold out</span>}
    </div>
  );
}
```

4.3 Lists and Conditional Rendering

Use array.map(...) to turn data into elements. Every item needs a stable, unique **key** (use the database id, not the array index when the list can change). Use condition ? A : B or condition && A for conditional UI.

```
import ProductCard from './components/ProductCard.jsx';

const products = [
  { id: 1, name: 'Keyboard', price: 25, inStock: true },
  { id: 2, name: 'Mouse', price: 12, inStock: false },
  { id: 3, name: 'Monitor', price: 180, inStock: true },
];

export default function App() {
  return (
    <div>
      <h1>Products</h1>
      {products.map(p => (
        <ProductCard key={p.id} name={p.name} price={p.price} inStock={p.inStock} />
      ))}
    </div>
  );
}
```

5. State with useState

A **hook** is a special function whose name starts with use. Hooks can only be called at the top level of a component (never inside if, loops or nested functions).

const [value, setValue] = useState(initialValue) gives you the current value and a function to change it. Calling the setter schedules a re-render with the new value.

5.1 Counter

Re-read the counter from Section 2.1. Then try setCount(count + 1) three times in one click handler: the result is +1, not +3, because count is the same snapshot in all three calls. When the new value depends on the old one, use the functional form: setCount(c => c + 1).

5.2 Controlled Inputs

The input's value lives in state, so React is always the single source of truth.

```
const [keyword, setKeyword] = useState('');

<input
  value={keyword}
  onChange={e => setKeyword(e.target.value)}
  placeholder="Search..."
/>
<p>You typed: {keyword}</p>
```

5.3 State That Holds Arrays and Objects

Never modify state directly. Always create a **new** array or object, so React can detect the change.

```
const [items, setItems] = useState([]);

// add
setItems([...items, newItem]);
// remove
setItems(items.filter(i => i.id !== id));
// update one
setItems(items.map(i => (i.id === id ? { ...i, done: !i.done } : i)));

// WRONG – mutates the existing array, React does not notice
items.push(newItem);
setItems(items);
```

> **Warning**
> The setter does not change the variable immediately: setCount(5); console.log(count); still prints the old value. The new value appears on the next render.

6. Side Effects with useEffect

Rendering must be a pure calculation. Anything else – fetching data, timers, subscriptions, changing document.title – is a **side effect** and belongs in useEffect, which runs *after* React has updated the screen.

```
useEffect(() => {
  document.title = `Clicked ${count} times`;
}, [count]);          // runs after the first render and whenever count changes
```

6.1 The Dependency Array

```
useEffect(() => { ... });        // no array  -> after EVERY render (rarely wanted)
useEffect(() => { ... }, []);    // empty []  -> once, after the first render
useEffect(() => { ... }, [id]);  // [id]      -> first render + whenever id changes
```

6.2 Cleanup

If an effect starts something that keeps running (a timer, a socket, an event listener), return a cleanup function that stops it. React runs the cleanup before the effect runs again and when the component is removed.

```
function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const id = setInterval(() => setSeconds(s => s + 1), 1000);
    console.log('timer started');
    return () => {                // cleanup: runs when the component disappears
      clearInterval(id);
      console.log('timer stopped');
    };
  }, []);

  return <p>{seconds}s</p>;
}

// parent
const [show, setShow] = useState(true);
<button onClick={() => setShow(!show)}>Toggle</button>
{show && <Timer />}
```

> **Note**
> In development, <StrictMode> deliberately runs every effect **twice** (start → cleanup → start) to help you find missing cleanups. In the console you will therefore see “timer started”, “timer stopped”, “timer started” when the component first appears. The same is true for data fetching: DevTools → Network shows two requests, and the first one is cancelled by the cleanup. This does not happen in the production build.

6.3 Common Mistakes

Mistake	Consequence	Fix
Forgetting the [] array on an effect that sets state	Infinite loop of render → effect → setState → render	Add the dependency array
Missing a variable that the effect uses in its array	Effect uses stale values	List every reactive value the effect reads
async function directly as the effect callback	Effect must return a cleanup function, not a Promise	Define an inner async function and call it, or use .then()

Mistake	Consequence	Fix
Calling setState in the component body	“Too many re-renders”	Call setters only in event handlers or effects

7. Other Hooks You Will Meet

Hook	Use it when	Remember
useRef	You need to reach a DOM element or keep a value between renders **without** re-rendering	Changing ref.current does not trigger a render
useContext	Many components need the same data (logged-in user, theme)	Avoids passing props through every level (“prop drilling”)
useMemo / useCallback	Optimizing expensive calculations / stable function identities	Not needed at this stage; measure first
custom hooks	The same stateful logic appears in several components	Plain functions whose name starts with use – see Section 8.3

8. Fetching Data from an API

Real applications rarely hard-code their data – they ask a server for it over HTTP. In the browser the built-in fetch() function sends the request and returns a *Promise*. Fetching is a side effect, so it belongs in useEffect. Every request has three possible states, and your UI must show each of them: **loading**, **error** and **success**.

All examples below use **DummyJSON** (https://dummyjson.com), a free public API that needs no key and allows requests from the browser. Open https://dummyjson.com/products?limit=2 in your browser first to see what the JSON looks like.

8.1 Fetching in useEffect: Loading, Error, Success

```
import { useEffect, useState } from 'react';

export default function ProductList() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
```

```
  useEffect(() => {
    const controller = new AbortController();

    async function load() {          // the effect itself cannot be async
      try {
        const res = await fetch('https://dummyjson.com/products?limit=10',
                                { signal: controller.signal });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const data = await res.json(); // { products, total, skip, limit }
        setProducts(data.products);
      } catch (err) {
        if (err.name !== 'AbortError') setError(err.message);
      } finally {
        if (!controller.signal.aborted) setLoading(false);
      }
    }

    load();
    return () => controller.abort();  // cleanup: cancel the request
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p style={{ color: 'red' }}>Error: {error}</p>;

  return (
    <ul>
      {products.map(p => <li key={p.id}>{p.title} – ${p.price}</li>)}
    </ul>
  );
}
```

Rule	Why
Check res.ok	fetch only rejects on network failure. A 404 or 500 response is still “successful”, so you must throw yourself.
await res.json()	Reading the body is also asynchronous and returns the parsed JavaScript object.
Abort in the cleanup	If the component disappears or the dependencies change before the response arrives, the old request is cancelled and cannot overwrite newer data.
Ignore AbortError	A cancelled request is not a real error – do not show it to the user.

Rule	Why
Put inputs in the dependency array	If the URL depends on state (page, keyword, id), list that state in the array so the effect fetches again when it changes.

8.2 Keep API Calls in a Service File

Components should not know URLs. Put every request in src/services/ and export one small function per endpoint. If the server address changes, you edit one file.

```
// src/services/api.js
const BASE_URL = import.meta.env.VITE_API_URL ?? 'https://dummyjson.com';

async function request(path, options = {}) {
  const res = await fetch(BASE_URL + path, {
    ...options,
    headers: { 'Content-Type': 'application/json', ...options.headers },
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}

export const getProducts = (limit, skip, signal) =>
  request(`/products?limit=${limit}&skip=${skip}`, { signal });
export const getProduct = id => request(`/products/${id}`);
export const createProduct = data =>
  request('/products/add', { method: 'POST', body: JSON.stringify(data) });
export const updateProduct = (id, data) =>
  request(`/products/${id}`, { method: 'PUT', body: JSON.stringify(data) });
export const deleteProduct = id =>
  request(`/products/${id}`, { method: 'DELETE' });
```

8.3 A Custom Hook: useFetch

Loading / error / data logic is the same in every screen. Move it into a *custom hook* – a normal function whose name starts with use and that calls other hooks. Each component that calls it gets its own independent state.

```
// src/hooks/useFetch.js
import { useEffect, useState } from 'react';

export function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    setLoading(true);
    setError(null);
    fetch(url, { signal: controller.signal })
      .then(res => {
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json();
      })
      .then(json => { setData(json); setLoading(false); })
      .catch(err => {
        if (err.name !== 'AbortError') { setError(err.message); setLoading(false); }
      });
    return () => controller.abort();
  }, [url]);                    // fetch again whenever the URL changes

  return { data, loading, error };
}

// usage in any component
const { data, loading, error } = useFetch('https://dummyjson.com/products?limit=5');
```

Note

Write operations on DummyJSON (POST, PUT, DELETE) are *simulated*: the server answers as if it succeeded and returns the resulting object, but nothing is saved. Your app must therefore update its own state with the returned object. After a page reload, the changes are gone – this is expected.

Use the browser DevTools → **Network** tab to see every request, its status code and its JSON response. It is the fastest way to debug API problems.

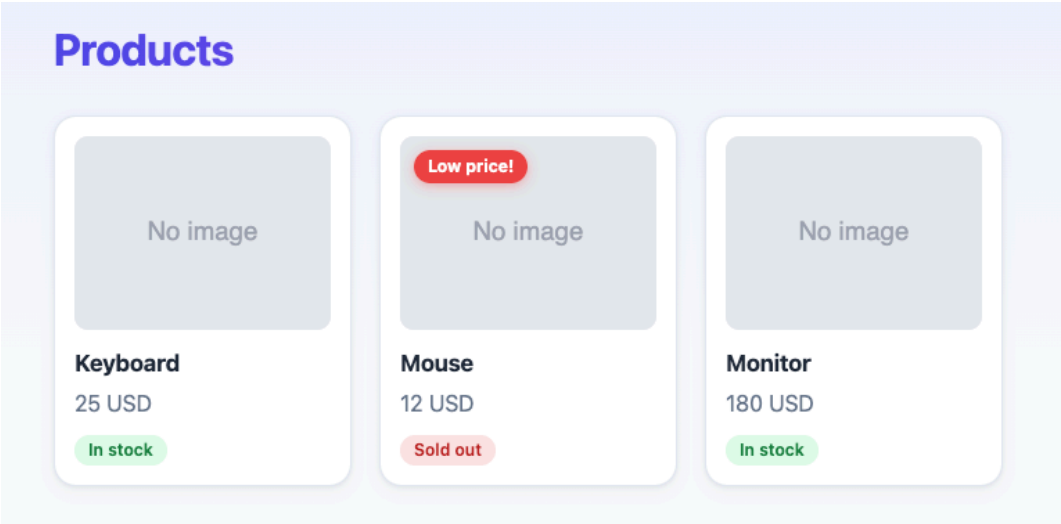
9. In-Class Exercises

Exercise 1: Components and Props

Goal: Build a reusable component and render a list.

1. Create ProductCard (name, price, inStock) and render the 3 sample products from Section 4.3.

2. Show a red “Low price!” badge when price < 20. Add an image prop with a default placeholder.



Sample code:

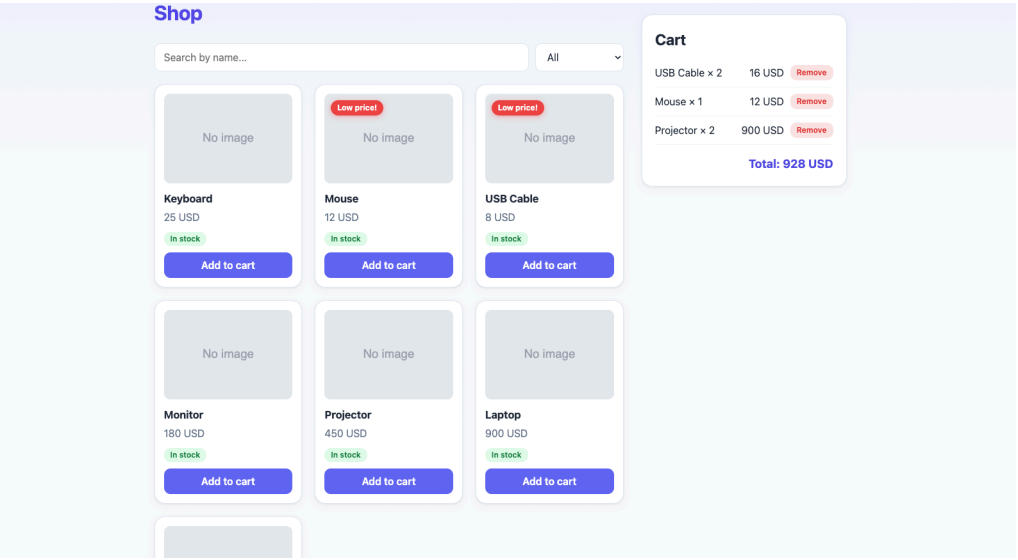
```jsx
export default function ProductCard({
  name,
  price,
  inStock = true,
  image = PLACEHOLDER,
}) {
  return (
    <div className="card">
      {price < 20 && <span className="badge">Low price!</span>}
      <img src={image} alt={name} />
      <h3>{name}</h3>
      <p>{price.toLocaleString()} USD</p>
      {inStock ? (
        <span className="stock in">In stock</span>
      ) : (
        <span className="stock out">Sold out</span>
      )}
    </div>
  );
}
```

Exercise 2: Product List and Shopping Cart (your own data)

Goal: Practice useState and useEffect on a data set you define yourself.

Setup: In src/data/products.js, export an array of at least 6 products (id, name, price, category) in 2–3 categories. Reuse ProductCard from Exercise 1.

1. Add a search box (by name, case-insensitive) and a category dropdown to filter the list.
2. Add an "Add to cart" button. Adding the same product again increases its quantity. Show the cart with a "Remove" button and the total price.
3. Use useEffect to show the number of cart items in document.title.
4. Load the products through fakeFetchProducts() and show "Loading..." until the data arrives.



Sample code:

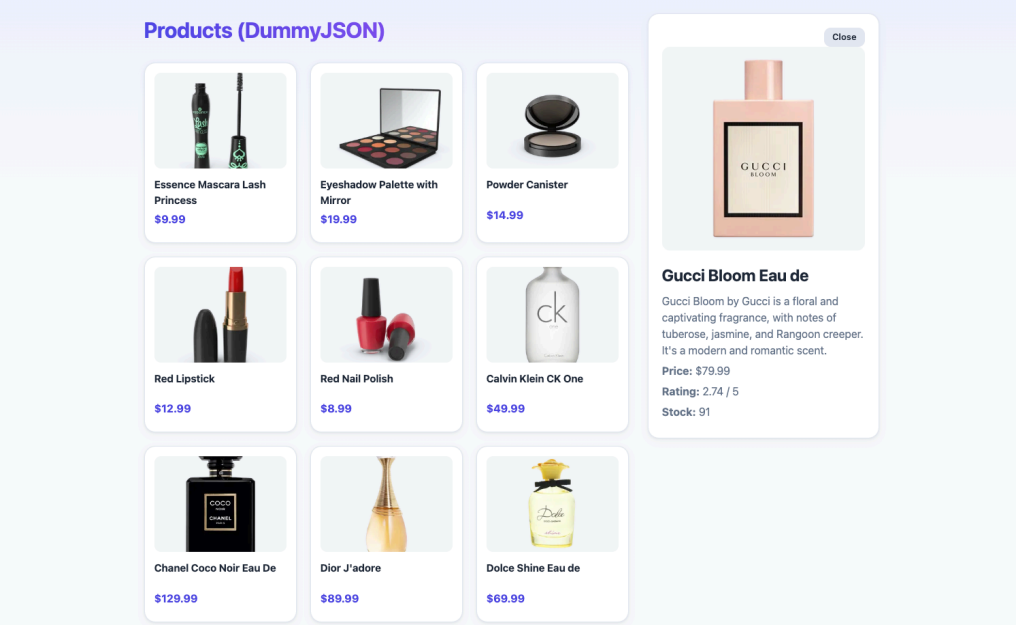
```
export const products = [
  { id: 1, name: 'Keyboard', price: 25, category: 'Accessories', inStock: true },
  { id: 2, name: 'Mouse', price: 12, category: 'Accessories', inStock: true },
  { id: 3, name: 'USB Cable', price: 8, category: 'Accessories', inStock: true },
  { id: 4, name: 'Monitor', price: 180, category: 'Displays', inStock: true },
  { id: 5, name: 'Projector', price: 450, category: 'Displays', inStock: true },
  { id: 6, name: 'Laptop', price: 900, category: 'Computers', inStock: true },
  { id: 7, name: 'Mini PC', price: 350, category: 'Computers', inStock: true },
];

// Simulates a network request: resolves with the products after 1 second.
export function fakeFetchProducts() {
  return new Promise(resolve => {
    setTimeout(() => resolve(products), 1000);
  });
}
```

Exercise 3: Real Data from a Public API

Goal: Replace your own data with real data from **DummyJSON** (https://dummyjson.com, free, no API key). Read Section 8 first.

1. Create src/services/productApi.js. Load GET /products?limit=12&skip=0 and display the products (fields: title, price, thumbnail). Show "Loading..." and an error message when the request fails.
2. Add "Previous / Next" buttons: skip = (page - 1) * 12. Disable them on the first / last page (use total).
3. Click a product to show its details from GET /products/{id} (description, rating, stock).



10. Homework

Homework: Mini Blog Manager

Build a React app that manages blog posts from the public API https://dummyjson.com/posts. Organize the code into components/, pages/, hooks/ and services/.

Endpoint	Returns
GET /posts?limit=10&skip=0	{ posts, total, skip, limit } – each post has id, title, body, tags, reactions

Endpoint	Returns
GET /posts/search?q=love	Posts that contain the keyword
GET /posts/{id}/comments	{ comments: [{ body, user: { username } }] }
POST /posts/add	Body { title, body, userId: 1 } → the new post
DELETE /posts/{id}	The deleted post with isDeleted: true

Requirements

- **List:** show 10 posts per page (title, tags, likes) with "Previous / Next", a loading indicator and an error message.
- **Search:** a search box that calls /posts/search 400 ms after the user stops typing (custom hook useDebounce, see below).
- **Detail:** click a post to see its full body and its comments.
- **Add and delete:** a form (title, body, both required) that calls POST /posts/add, and a "Delete" button with confirmation. Update the list in state using the server's response.

> **Warning**
> DummyJSON does not really save POST / DELETE – it only returns the result. That is why you must update your own state; after a reload the changes are gone, which is expected.

